Metaheuristic Algorithms and Applications: Program Report I

第 17 組

組員： N26140804 張暐俊

1. Introduction

這次作業選擇實作並比較 **Genetic Algorithm (GA)** 與 **Particle Swarm Optimization (PSO)**，主要是因為這兩個演算法在課程中都有講解，也有提供相關範例程式，實作上比較熟悉。實驗使用作業指定的七個 benchmark functions，目標是找出各函數的全域最小值 x^* 與對應的 fitness 值 F^* ，並比較兩個演算法的收斂行為與最佳化效果。

2. Benchmark Functions

2.1 Unimodal

Function	Dim	Search Range	Optima
Schwefel 2.22	2	$[-10, 10]^2$	0
Step	2	$[-100, 100]^2$	0

Schwefel 2.22:

$$F_2(x) = \sum_{i=1}^{D} |x_i| + \prod_{i=1}^{D} |x_i|$$

Step

$$F_6(x) = \sum_{i=1}^{D} (\lfloor x_i + 0.5 \rfloor)^2$$

其中 step function 最佳解並非單點，而是整個 $[-0.5,0.5]^2$ ，所以任何落在此範圍內的點都會被 *floor(x+0.5)* 四捨五入為 0，fitness 等於 0。

2.2 Multimodal (fixed dimension)

Function	**Dim**	**Search Range**	**Optima**
Rastrigin	2	$[-5.12, 5.12]^2$	0
Griewank	2	$[-600, 600]^2$	0
Penalized	2	$[-50,\ 50]^2$	0
Kowalik	4	$[-5, 5]^4$	0.0003075
Branin	2	$[-5,10] \times [0,15]$	0.398

Rastrigin

$$F_9(x) = \sum_{i=1}^{D} [x_i^2 - 10\cos(2\pi x_i) + 10]$$

Griewank

$$F_{11}(x) = \frac{1}{4000} \sum_{i=1}^{D} x_i^2 - \prod_{i=1}^{D} \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

Penalized

$$F_{13}(x) = 0.1\left\{\sin^2(3\pi x_1) + \sum_{i=1}^{D-1}(x_i - 1)^2[1 + \sin^2(3\pi x_{i+1})]\right.$$

$$\left. + (x_D - 1)^2[1 + \sin^2(2\pi x_D)]\right\} + \sum_{i=1}^{D} u(x_i, 5,100,4)$$

$$u(x_i, a, k, m) = \begin{cases} k(x_i - a)^m, & x_i > a \\ 0, & -a \le x_i \le a \\ k(-x_i - a)^m, & x_i < a \end{cases}$$

全域最佳解在 $x^* = (1,1)$ 而非原點；懲罰項 $\$u(x, 5,100,4) = 100(|x| - 5)^4$ 推估在邊界附近會爆炸。

Kowalik

$$F_{15}(x) = \sum_{i=1}^{11}\left[a_i - \frac{x_1(b_i^2 + b_i x_2)}{b_i^2 + b_i x_3 + x_4}\right]^2$$

Branin

$$F_{17}(x) = \left(x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5}{\pi}x_1 - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)\cos x_1 + 10$$

Branin 有 3 個全域最佳解，fitness 值相同：

- $(-\pi,\ 12.275)$
- $(\pi,\ 2.275)$
- $(9.42478, 2.475)$

在計算 distance error 時取距離最近者。

3. Programs

接下來會簡單地分別說明一下關於兩個演算法的程式碼撰寫與設計。其中兩個演算法都以老師放在 Moodle 上的範例程式（以下稱 *GA program* 與 *PSO program*）為主，並配合作業需求做了調整。

3.1 Genetic Algorithm (GA)

GA program 使用 binary encoding，用 12 bits 表示一個 1D 實數。但這次有 7 個函數，維度從 2D 到 4D，搜尋範圍也各不相同（Branin 的 x_1, x_2 範圍還不對稱），若沿用 binary encoding 就需要為每個維度分別設定 bit 數和解碼比例，實作麻煩且容易出錯。改用 real-valued encoding 後，位置直接就是一個浮點數陣列，天生支援任意維度和非對稱邊界。

GA program 用 roulette wheel selection，但 roulette wheel 需要 fitness 越大越好，所以老師加了一層轉換才能用。這次改用 tournament selection（

$k = 3$），不需要任何轉換，直接在原始 fitness 上比大小就好。另外 roulette wheel 在族群裡 fitness 差距懸殊時容易幾乎只選到少數幾個個體，tournament selection 因為只比相對大小，不會有這個問題。

另一個改動則是加入了 elitism 。 *GA program* 沒有這個機制，這會導致下一代的 crossover 或 mutation 有機會把好不容易找到的最佳解改掉，讓收斂不穩定。這次把當前最佳個體直接保留到下一代，確保每代不會退步。

以上都可以在主迴圈裡看到：

```
for gen in range(max_iter):
    new_pop = np.empty_like(pop)
    new_pop[0] = best_pos.copy()  # elitism

    i = 1
    while i < pop_size:
        parent1 = _tournament_select(pop, fitness, tournament_size, rng)
        parent2 = _tournament_select(pop, fitness, tournament_size, rng)

        if rng.random() < crossover_rate:
            child1, child2 = _blx_crossover(parent1, parent2, alpha, lower, upper, rng)
```

```
        else:
            child1, child2 = parent1.copy(), parent2.copy()

        child1 = _mutate(child1, mutation_rate, mutation_scale, lower, upper, rng)
        child2 = _mutate(child2, mutation_rate, mutation_scale, lower, upper, rng)

        new_pop[i] = child1; i += 1
        if i < pop_size:
            new_pop[i] = child2; i += 1

    pop = new_pop
    fitness = np.array([func(ind) for ind in pop])

    gen_best_idx = np.argmin(fitness)
    if fitness[gen_best_idx] < best_val:
        best_val = fitness[gen_best_idx]
        best_pos = pop[gen_best_idx].copy()
```

其中 *_tournament_select* 從族群隨機抽 k 個個體取 fitness 最低者：

```
def _tournament_select(pop, fitness, k, rng):
    indices = rng.choice(len(pop), size=k, replace=False)
    winner = indices[np.argmin(fitness[indices])]
    return pop[winner].copy()
```

而 *GA program* 的 crossover 是對 binary bits 做單點交叉，在實數空間沒有意義。這次改用 BLX-α（$\alpha = 0.5$，rate = 0.8），子代從兩個 parent 的延伸區間內均勻取樣：

$$[\min(p_1, p_2) - \alpha|p_1 - p_2|,\ \max(p_1, p_2) + \alpha|p_1 - p_2|]$$

$\alpha = 0.5$ 讓子代有機會落在 parent 連線外側，不只是在兩者之間取樣，保留了往外探索的空間。

```
def _blx_crossover(p1, p2, alpha, lower, upper, rng):
    d = np.abs(p1 - p2)
    lo = np.minimum(p1, p2) - alpha * d
    hi = np.maximum(p1, p2) + alpha * d
    child1 = np.clip(rng.uniform(lo, hi), lower, upper)
    child2 = np.clip(rng.uniform(lo, hi), lower, upper)
```

```
    return child1, child2
```

最後在 mutation 的部分， *GA program* 的是 bit flip，同樣不適用於實數。所以改用 Gaussian perturbation，每個基因獨立以機率 0.1 受到擾動。σ 設為搜尋範圍的 10%，這樣不同 range 的函數不需要個別調整，Griewank 的 $[-600, 600]$ 和 Schwefel 的 $[-10, 10]$ 自然等比例。

```
def _mutate(ind, rate, scale, lower, upper, rng):
    mask = rng.random(len(ind)) < rate
    if np.any(mask):
        sigma = scale * (upper - lower)
        ind[mask] += rng.normal(0, sigma[mask])
        ind = np.clip(ind, lower, upper)
    return ind
```

當前 GA 的整體風格是靠族群多樣性運作——每代族群仍散佈在搜尋空間各處，靠 elitism 帶著最佳解慢慢前進。這和接下來設計的 PSO 截然不同。

3.2 Particle Swarm Optimization (PSO)

PSO program 的速度更新中，前一代速度直接疊加，沒有遞減機制。這次加入了線性遞減的慣性權重 w：

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 \left(p_{best,i} - x_i^t\right) + c_2 r_2 (g_{best} - x_i^t)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

前期 w 大（0.9），粒子保留較多舊速度，偏向廣域探索；後期 w 縮到 0.4，粒子速度快速衰減，集中收斂。w 使用 $t/(T-1)$ 而非 t/T ，確保最後一輪精確降到 $w_{min} = 0.4$：

$$w(t) = w_{max} - (w_{max} - w_{min}) \cdot \frac{t}{T-1}$$

PSO program 也沒有速度限制，粒子可能一步飛出搜尋範圍。所以加入 $v_{max} = 0.2 \times \text{range}$，讓單步位移不超過搜尋範圍的 20%。

另外 *PSO program* 的 r_1, r_2 是純量，所有維度共用同一個隨機值。這次改成每個維度各自取樣，讓各維度的更新有各自的隨機性。完整的迭代迴圈如下：

```
for t in range(max_iter):
    w = w_max - (w_max - w_min) * t / (max_iter - 1)

    for i in range(pop_size):
        r1 = rng.random(D)
        r2 = rng.random(D)

        vel[i] = (w * vel[i]
                  + c1 * r1 * (pbest_pos[i] - pos[i])
                  + c2 * r2 * (gbest_pos - pos[i]))
        vel[i] = np.clip(vel[i], -v_max, v_max)

        pos[i] = np.clip(pos[i] + vel[i], lower, upper)
        fitness[i] = func(pos[i])

        if fitness[i] < pbest_val[i]:
            pbest_val[i] = fitness[i]
            pbest_pos[i] = pos[i].copy()
        if fitness[i] < gbest_val:
            gbest_val = fitness[i]
            gbest_pos = pos[i].copy()
```

位置先 clip 再評估 fitness，避免讓出界的位置進入 pbest 或 gbest。

那相對於前述的 GA 靠族群多樣性慢慢前進，PSO 透過 g_{best} 把整個粒子群往好解集中，是截然不同的收斂策略。

3.3 Experimental Framework

實驗由 *run_experiment.py* 統一跑，結果存進 *experiment_results.json*，再由 *plot_results.py* 讀取產生圖表與表格。

兩個演算法各跑七個函數、各跑 100 和 1000 兩種 iter，每組設定獨立執行 50 次（seed 0–49），合計 1400 次。

```python
for algo_name, algo_func in algorithms.items():       # GA, PSO
    for func_name, func_info in FUNCTIONS.items():    # 7 functions
        for max_iter in [100, 1000]:
            run_experiment(algo_name, algo_func, func_name, func_info, max_iter, num_runs=50)
```

每組跑完後收集 best fitness 的平均、標準差、中位數（Table 4、5），所有 runs 最後一代族群的平均 fitness（*avg_mean_fitness*），以及 distance error $\|x_{best} - x^*$ ——對有多個最佳解的函數（Branin）取距離最近者。另外把 50 條收斂曲線逐 iteration 取平均，留給 **Section 5.3** 畫圖用。

為節省空間，每組只有 run #0 會額外保存完整的搜尋軌跡和最後一代族群位置，作為 **Section 5.2 sample run** 圖的來源。

4. Parameter Settings

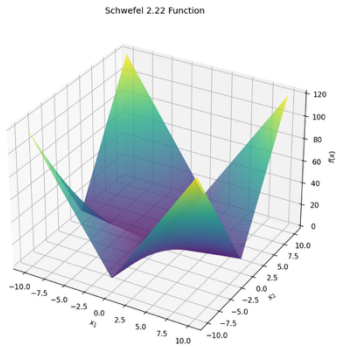
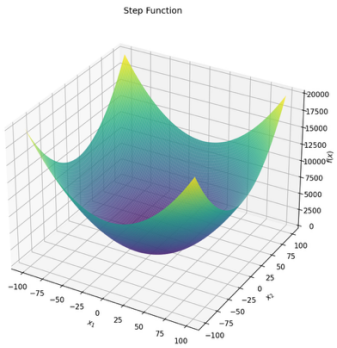
Parameter	**GA**	**PSO**
Population size	50	50
Max iterations	100 / 1000	100 / 1000
Number of runs	50	50
Selection	Tournament ($k = 3$)	—
Crossover	BLX-α ($\alpha = 0.5$, rate $= 0.8$)	—
Mutation	Gaussian ($\sigma = 0.1 \times$ range, rate $= 0.1$)	—
Elitism	Yes (best preserved)	—
Inertia weight w	—	$0.9 \rightarrow 0.4$ (linear)
Cognitive c_1	—	2.0

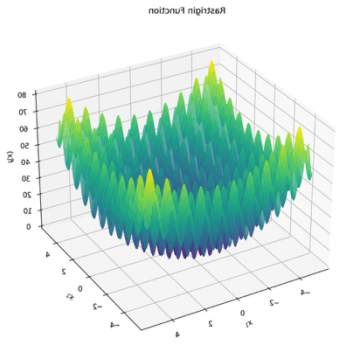
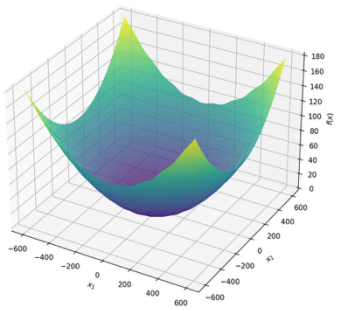
Social c_2	—	2.0
Velocity clamp v_{max}	—	0.2 × range

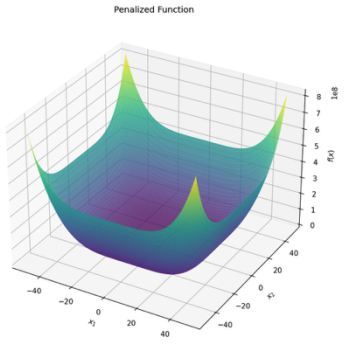
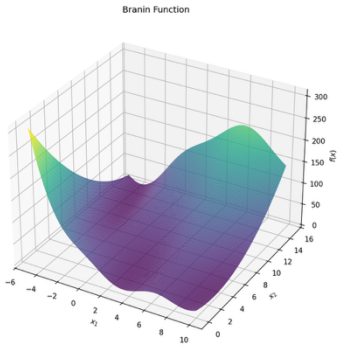
5. Discussions

5.1 3D Surface Plots (D = 2)

六個 2D benchmark functions 的 function landscape 如下，其中 Kowalik 為 4D 函數，所以沒有 3D 的 surface：

Schwefel 2.22	**Step**
	

Rastrigin	**Griewank**
	

Penalized	Branin
	

可以觀察到以下幾點：

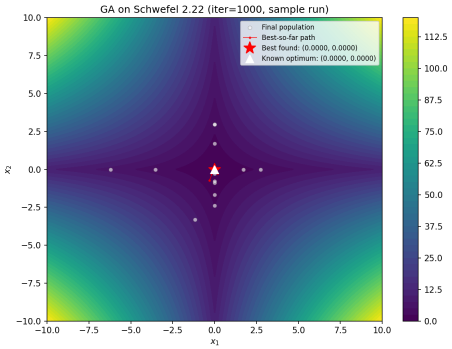
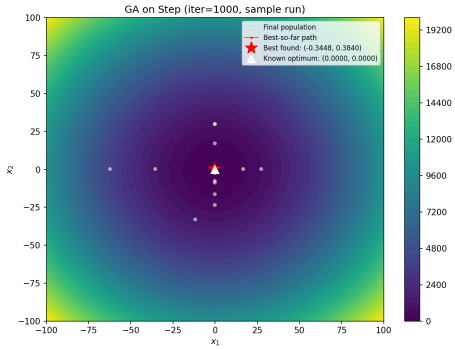
- **Schwefel 2.22** 是典型的單峰凸函數，朝原點單調下降。
- **Step** 呈階梯狀，大片區域 fitness 相同，這讓梯度法無用武之地，但對 metaheuristic 演算法反而有利。
- **Rastrigin** 有規則排列的深谷，每個整數座標附近都是 local minimum。
- **Griewank** 在 $[-600,600]$ 的大範圍下看起來像光滑的碗，但局部有大量細微震盪。
- **Penalized** 多數區域平坦，邊界附近懲罰項快速爬升。
- **Branin** 有三個幾乎等高的全域最佳解，三個 basin 之間由稜線分隔。

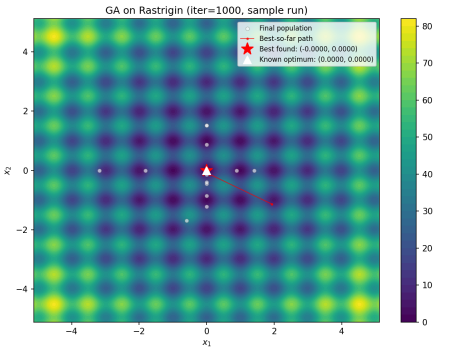
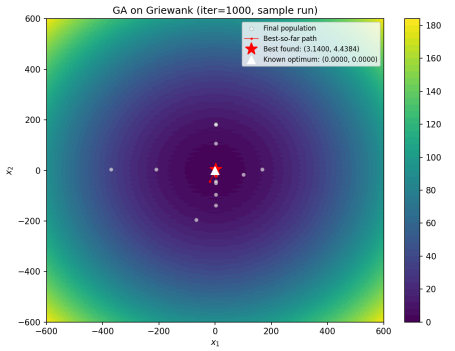
5.2 Search Results (Sample Run)

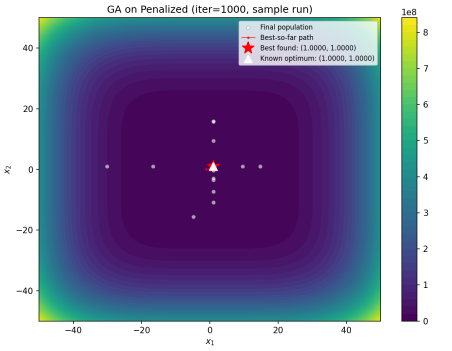
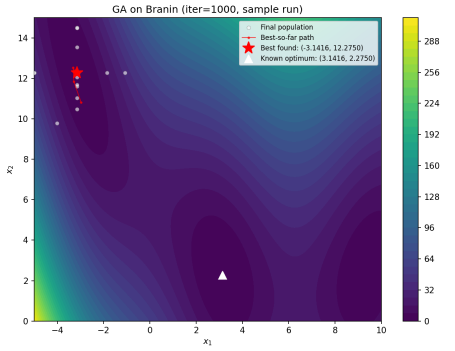
以 seed = 0、max_iter = 1000 的 sample run 呈現每個演算法在 2D 函數上的搜尋過程。以下圖例在圖中分別代表：

- **等高線背景**是 function landscape
- **白點**代表最後一代族群分佈
- **紅色軌跡**表示 best-so-far 隨 iteration 的移動路徑
- **紅星**為最終找到的 best 解
- **白三角**是理論已知最佳解 x^*

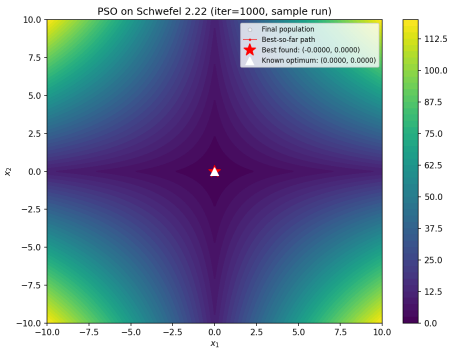
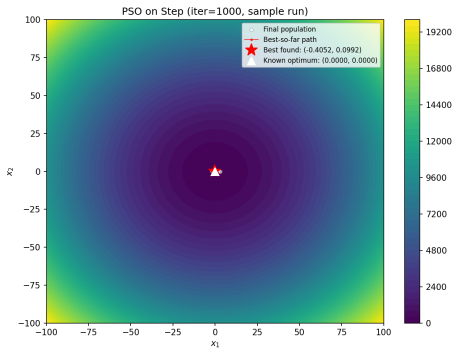
GA

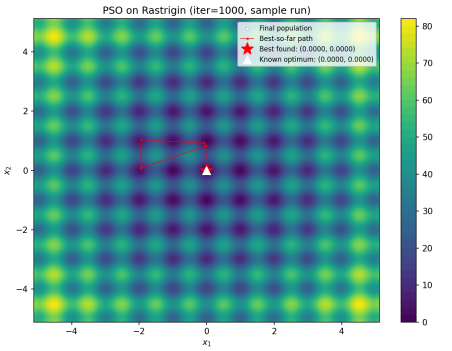
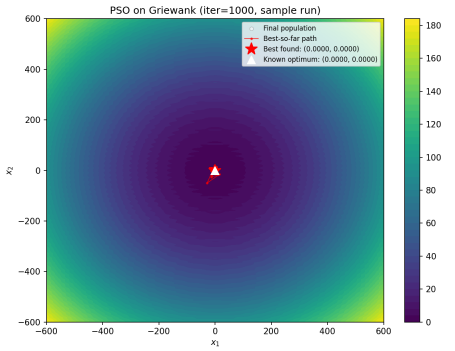
Schwefel 2.22	Step
	

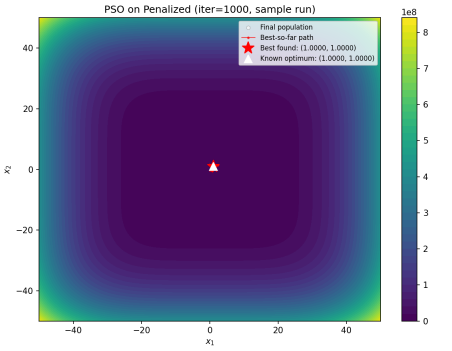
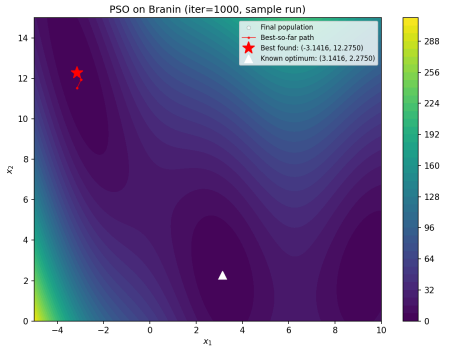
Rastrigin	Griewank
	

Penalized	Branin
	

PSO

Schwefel 2.22	Step
	

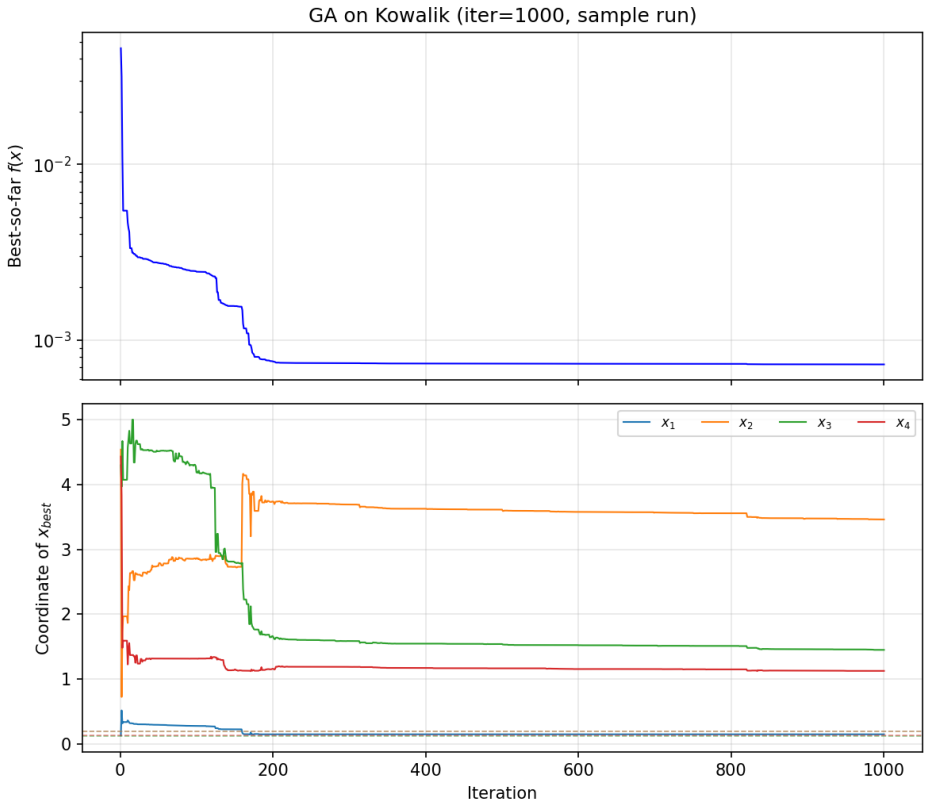
Rastrigin	Griewank
	

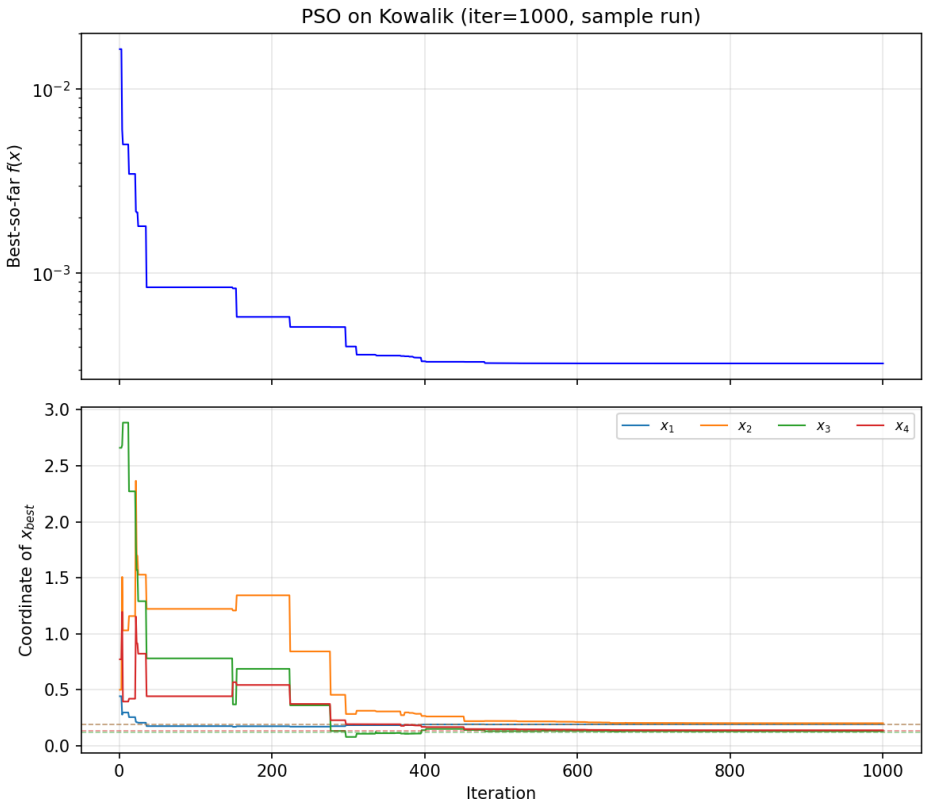
Penalized	Branin
	

從圖中可以看出兩個演算法的最後一代族群分佈風格截然不同。PSO 粒子會被 g_{best} 拉到最佳解周圍形成緊密群聚；GA 由於 crossover 與 mutation 每代都在重新組合族群，最後一代個體仍散佈於搜尋空間各處。而這個視覺差異也直接呼應了後面 Table 中 *Avg Mean Fitness* 欄位 GA 遠高於 PSO 的現象。

Kowalik (4D)

Kowalik 為 **4** 維函數，無法以 **2D** 等高線的形式呈現搜尋軌跡，因此改以兩張圖表示單一次 **run** 的搜尋過程。圖為 **best-so-far f(x)** 隨 **iteration** 的收斂曲線，下圖為 $\mathbf{x_{best}}$ 四個座標各自的演化軌跡，虛線則是對應維度的理論最佳解位置。

GA on Kowalik


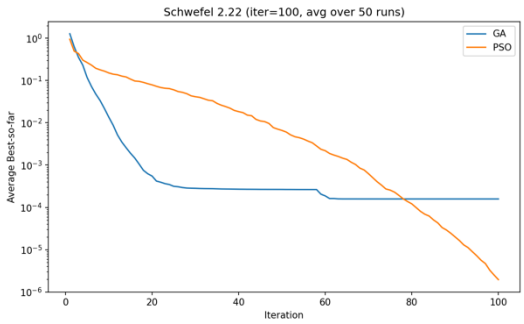
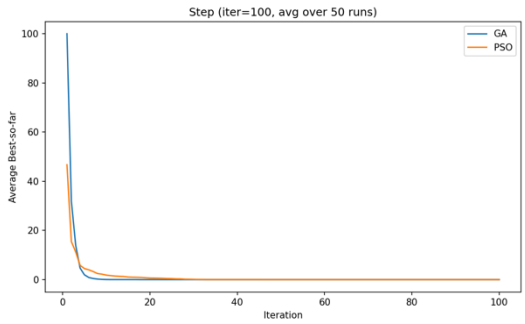
PSO on Kowalik


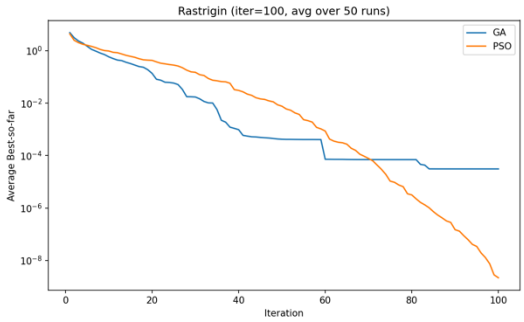
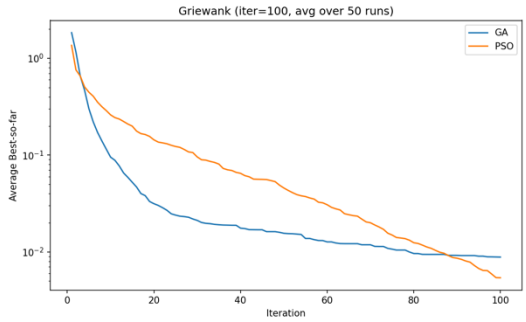
PSO 的每個座標都能緩慢貼近 $\mathbf{x}^*$ 的虛線，而 GA 的 $\mathbf{x_2}, \mathbf{x_3}, \mathbf{x_4}$ 在 iter 數百之後仍在原地震盪，fitness 也相應地停在較高的 plateau，和 Table 6 中 Kowalik GA distance error 遠大於 PSO 的結果一致。

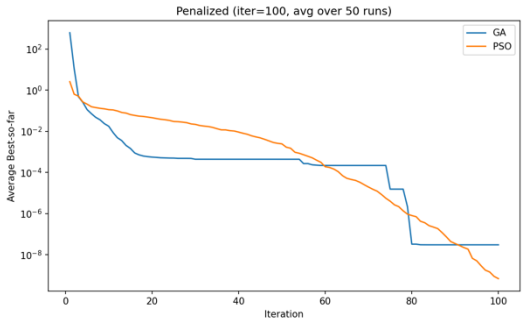
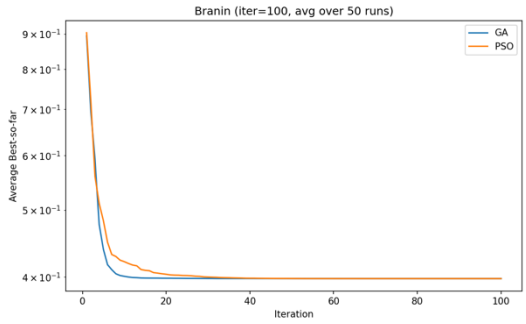
5.3 Convergence Curves

每張圖在同一坐標系下對比 GA 與 PSO 的平均 best-so-far vs iteration（取 50 runs 平均）。當所有值嚴格大於 0 時使用 log scale，便於觀察收斂後段的細部差異。

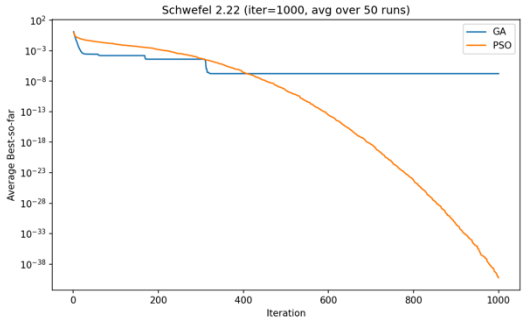
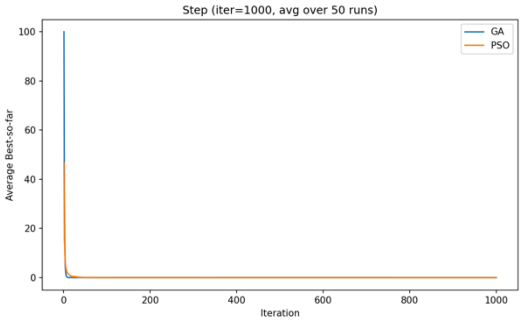
iter = 100

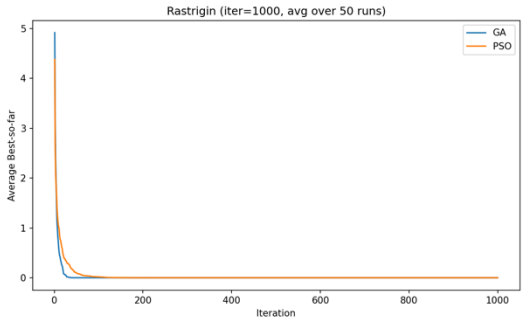
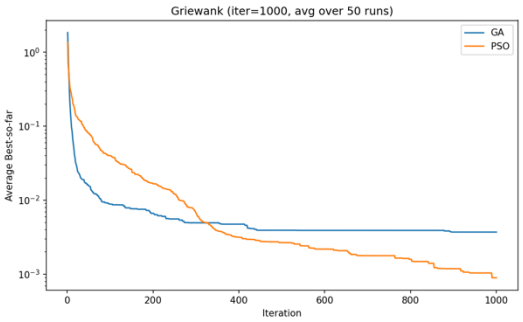
Schwefel 2.22	Step
	

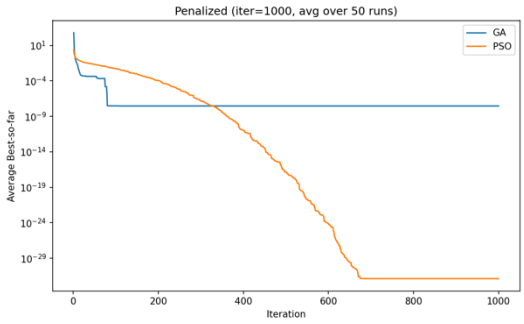
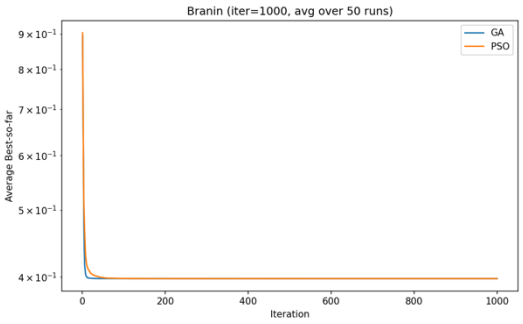
Rastrigin	Griewank
	

Penalized	Branin
	

iter = 1000

Schwefel 2.22	Step
	

Rastrigin	Griewank
	

Penalized	Branin
	

從收斂曲線來看，PSO 不一定一開始就比 GA 快，但它不會停。以 Schwefel 2.22 為例，前期反而是 GA 先衝下去，到 iter 400 左右 PSO 才追上，但之後繼續下探到幾乎機器精度，GA 則大約 iter 350 後就停住了。Rastrigin 和

Penalized 也是類似的狀況，1000 iter 時 PSO 都降到數值零，GA 仍然停在 10^{-8} 附近。這背後應該是 g_{best} 的吸引力在作用——粒子一旦集中過去就會持續細調；反觀 GA 每代都在重新洗牌，很難一直聚焦在同一個方向推進。

另外，多跑到 1000 iter 的效果相當不平均。對 Schwefel 2.22、Rastrigin、Branin 這類比較簡單的函數，100 iter 大致就夠了；但對 Griewank 和 Kowalik，PSO 延長後還看得出持續進步（Griewank 的 distance error 從 3.06 降到 0.65），GA 幾乎沒動。感覺 GA 遇到困難函數不是時間不夠的問題，而是根本卡住出不來。

從函數難度來看，Unimodal 函數（Schwefel 2.22、Step）在前 20 iter 內就能收斂到接近最佳值；Rastrigin 雖是 multimodal，但 D = 2 加上規律排列的 basin，難度相對有限。真正困難的是 Griewank 與 Kowalik，兩者的收斂曲線在早期就進入緩慢遞減，顯示演算法陷入 local minima 後很難脫身。特別值得注意的是 Griewank 的「fitness 看起來小，但離最佳解其實很遠」：GA 在 1000 iter 的 *avg_best_so_far* 只有 0.0037，*distance_error_mean* 卻達 2.72，因為 Griewank 在大範圍內有大量接近零的 local minima，收斂到小 fitness 並不代表找到了真正的 $x^* = 0$。

5.4 Averaged Results (Table 4)

以下三張子表列出 50 runs 中最後一代的三個指標：平均 best-so-far、平均 mean fitness、中位數 best-so-far。每列的最小值以**粗體**標示（若全數並列則不標）。

Table 4a — Average Best-so-far

Function	GA (100)	GA (1000)	PSO (100)	PSO (1000)
Schwefel 2.22	1.58e-04	1.80e-07	1.95e-06	**0**
Step	0	0	0	0
Rastrigin	3.09e-05	1.00e-08	**0**	**0**
Griewank	8.85e-03	3.70e-03	5.41e-03	**8.92e-04**
Penalized	3.00e-08	3.00e-08	**0**	**0**
Kowalik	4.21e-03	3.59e-03	1.84e-03	**1.20e-03**
Branin	0.3979	0.3979	0.3979	0.3979

Table 4b — Average Mean Fitness

Function	GA (100)	GA (1000)	PSO (100)	PSO (1000)
Schwefel 2.22	0.3473	0.2846	7.29e-03	**0**
Step	78.51	66.62	0.9836	**0.8304**
Rastrigin	2.3108	1.9415	1.8522	**0**
Griewank	0.9413	0.7856	0.9632	**0.2757**
Penalized	1.49e+05	1.54e+05	1.74e-02	**0**
Kowalik	5.3317	5.0316	**0.8279**	100.73
Branin	1.4312	1.3117	1.2875	**0.5919**

Table 4c — Median Best-so-far

Function	GA (100)	GA (1000)	PSO (100)	PSO (1000)
Schwefel 2.22	**0**	**0**	1.39e-06	**0**
Step	0	0	0	**0**
Rastrigin	0	0	0	**0**
Griewank	7.40e-03	3.70e-03	7.40e-03	**0**
Penalized	0	0	0	**0**
Kowalik	1.38e-03	7.74e-04	6.98e-04	**3.24e-04**
Branin	0.3979	0.3979	0.3979	0.3979

Table 4b 裡比較有意思的現象是 GA 在 Step 和 Penalized 的 *Avg Mean Fitness* 遠高於 PSO。但這不是 GA 找不到好解，而是最後一代族群還散佈在搜尋空間各處，很多個體落在高 fitness 的區域把平均值拉上去了。特別是 Penalized，GA 的 *Avg Mean Fitnes* 衝到了 10^5 數量級，原因是懲罰項在邊界附近會爆炸式成長，只要幾個個體落到 $|x| > 5$ 的高懲罰區域就夠了。PSO 因為粒子都聚集在最佳解附近，*Avg Mean Fitness* 自然也小。

值得一提的是，Kowalik 的 PSO 在 1000 iter 的 *Avg Mean Fitness* 反而比 100 iter 更高。原因是 Kowalik 的分母 $b_i^2 + b_i x_3 + x_4$ 接近零時函數值會爆炸；延長迭代讓個別粒子有更多機會漂進病態區，即使 g_{best} 已收斂到最佳解附近，仍足以拉高族群平均。這暗示 Kowalik 對數值穩定性極敏感。

另外，Kowalik GA 100 的 median 和 avg 相差約三倍，說明 50 runs 裡有少數幾次卡得很深把平均拉上去了；Griewank 的 median 和 avg 就很接近，表示每次結果都差不多爛，不是偶爾才壞。關於各函數實際座標收斂到 x^* 的穩定度細節，見 **Table 5**。

5.5 Best Solution Statistics (Table 5)

以下呈現 50 runs 最後一代的 x_{best} 座標本身的平均 ± 標準差（逐維度列出）。由於不同函數的最佳解位置與維度不同，以每個函數一張子表的方式呈現，可直接對照第 2 節列出的 x^* 判讀演算法實際落點與穩定度。

Table 5a — Schwefel 2.22 ($x^* = (0,0)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
x_1	1.17e-04 ± 8.22e-04	1.02e-07 ± 7.32e-07	1.89e-07 ± 1.26e-06	-1.88e-41 ± 1.16e-40
x_2	4.06e-05 ± 2.84e-04	6.20e-08 ± 3.85e-07	-3.22e-07 ± 2.03e-06	5.39e-42 ± 5.80e-41

Table 5b — Step ($x^* \in [-0.5,0.5]^2$，以原點為基準)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
x_1	0.0922 ± 0.2812	0.0922 ± 0.2812	-0.0056 ± 0.2832	0.0703 ± 0.3229
x_2	0.0558 ± 0.2853	0.0558 ± 0.2853	0.0437 ± 0.2554	0.0671 ± 0.2774

Table 5c — Rastrigin ($x^* = (0,0)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
x_1	-1.50e-05 ± 1.12e-04	-1.25e-06 ± 7.23e-06	-2.41e-07 ± 2.21e-06	2.00e-10 ± 9.00e-10
x_2	-5.62e-05 ± 3.74e-04	6.92e-08 ± 1.20e-06	-6.08e-07 ± 2.38e-06	6.08e-11 ± 9.17e-10

Table 5d — Griewank ($x^* = (0,0)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
x_1	0.5700 ± 4.4309	0.4396 ± 2.1764	0.2476 ± 2.4981	0.1258 ± 1.0804
x_2	0.0865 ± 3.6007	0.0887 ± 3.1371	-0.3553 ± 3.3077	-5.58e-04 ± 1.5375

Table 5e — Penalized ($x^* = (1,1)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
$\boldsymbol{x_1}$	1.0000 ± 5.67e-05	1.0000 ± 5.67e-05	1.0000 ± 6.76e-06	1.0000 ± 0
$\boldsymbol{x_2}$	1.0000 ± 1.23e-04	1.0000 ± 8.37e-05	1.0000 ± 5.33e-05	1.0000 ± 0

Table 5f — Kowalik ($x^* \approx (0.1928,0.1908,0.1231,0.1358)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
$\boldsymbol{x_1}$	0.1216 ± 0.2197	0.0912 ± 0.1967	0.1301 ± 0.1347	0.1675 ± 0.1161
$\boldsymbol{x_2}$	1.6710 ± 3.4735	1.5525 ± 3.4519	1.7884 ± 2.6856	0.0824 ± 1.7696
$\boldsymbol{x_3}$	2.1294 ± 1.9162	1.2127 ± 1.5593	0.8438 ± 1.2760	0.0886 ± 1.0749
$\boldsymbol{x_4}$	1.5127 ± 1.7516	1.4167 ± 1.6946	0.9591 ± 1.2930	0.3134 ± 1.1461

Table 5g — Branin (三個全域最佳解：$(-\pi, 12.275), (\pi, 2.275), (9.425,2.475)$)

Dim	GA (100)	GA (1000)	PSO (100)	PSO (1000)
$\boldsymbol{x_1}$	3.6444 ± 3.9416	3.6442 ± 3.9420	2.2619 ± 3.7720	2.3876 ± 3.9017
$\boldsymbol{x_2}$	3.9225 ± 3.6451	3.9231 ± 3.6463	4.8990 ± 4.3726	4.9030 ± 4.3703

Table 5 比 **Table 4** 多給的資訊是座標本身有沒有對準 x^*，這在 **Table 5a**、**5c**、**5e** 上最清楚——Schwefel 2.22、Rastrigin、Penalized 的 PSO 1000 iter 座標 mean 都落在 x^*，std 也降到機器精度。**Table 4** 看到 fitness = 0 其實不能完全排除 Griewank 那種「離 x^* 很遠的 local minima 剛好值很小」的情況，但座標直接對上 x^* 就比較能確認真的找到了。

對照之下，**Table 5d** 的 Griewank 就是前面在擔心的狀況。PSO 1000 iter 的 $\$x_1, x_2$ mean 雖然逼近 0，但 std 還在 1 的數量級，比 mean 大好幾個量級，應該是大多數 runs 都收到原點、少數幾次被遠處的 local minima 卡住把 std 撐大了——也就是 **Table 6** 裡 Griewank distance error std 偏高的來源。

Table 5f 的 Kowalik 比較有意思，PSO 從 100 到 1000 進步特別明顯，x_2, x_3, x_4 都往 x^* 靠近；GA 給到 1000 iter 之後 x_2 還是停在 1.5 附近，跟 $x^* \approx 0.19$ 差了一個數量級。這大致就是 Section 5.2 Kowalik 單次 run 圖中 GA 座標後段原地震盪的統計版本。

另外 **Table 5g** 和 **5b** 的數字有點容易誤讀。Branin 三個 basin 位置差太遠，50 runs 落在不同 basin 平均起來會跑到三個之間的某個空地，mean 基本沒幾何意義，這欄還是要配 **Table 6** 的 distance error 看。Step 的 std 穩定在 0.28 ~ 0.32 不是演算法不穩，而是最佳解本來就是 $[-0.5, 0.5]^2$ 整個平台，平台內哪裡都一樣好，座標分散就是函數特性。

5.6 Distance Error (Table 6)

下表呈現的是兩個演算法找到的最佳解 x_{best} 與理論最佳解 x^* 之間的距離誤差，統計自 50 次獨立執行的平均值與標準差。Branin 則是取三個全域最佳解中距離最近者。其中每列最佳（mean 最小）以粗體標示。

Function	GA (100)	GA (1000)	PSO (100)	PSO (1000)
Schwefel 2.22	1.58e-04 ± 8.64e-04	0 ± 1.00e-06	2.00e-06 ± 2.00e-06	**0 ± 0**
Step	0.3929 ± 0.1332	0.3929 ± 0.1332	**0.3657 ± 0.1169**	0.4183 ± 0.1251
Rastrigin	1.05e-04 ± 3.81e-04	2.00e-06 ± 7.00e-06	2.00e-06 ± 2.00e-06	**0 ± 0**
Griewank	4.7708 ± 3.1890	2.7184 ± 2.7184	3.0594 ± 2.8299	**0.6530 ± 1.7666**
Penalized	3.60e-05 ± 1.33e-04	2.30e-05 ± 9.90e-05	3.50e-05 ± 4.10e-05	**0 ± 0**
Kowalik	4.8582 ± 1.8353	4.3088 ± 1.8396	3.1122 ± 2.1444	**0.9388 ± 2.1858**
Branin	1.76e-03 ± 5.99e-03	2.89e-04 ± 1.17e-03	2.00e-06 ± 2.00e-06	**0 ± 1.00e-06**

一個很有趣的觀察是 Step 的最佳解並非單點而是 $[-0.5, 0.5]^2$ 整個平台，任何落在此範圍內的點都有 fitness = 0。原點 (0,0) 本身就在平台內，理論下界為 0；對平台內的均勻分佈而言，到原點的期望距離約為 ≈ 0.38。實測值 GA 0.39、PSO 0.37 ~ 0.42 與此吻合，說明這是函數本身就無法提供更多梯度資訊的結果。也因此，Step 行中粗體標示的「PSO 100 勝出」本質上是噪音，沒有實際意義。

此外可以發現 Griewank 與 Kowalik 的 distance 遠大於 fitness 。例如 Griewank GA 1000 的 fitness 只有 3.70e-03（**Table 4a**），看起來非常接近 0，但 distance 高達 2.72。這是因為 Griewank 在 $[-600,600]$ 的 landscape 存在大量 local minima，在距離原點數個單位的地方就能有接近 0 的 fitness。單看 fitness 會嚴重低估問題難度，必須搭配 **Table 6** 一起判讀。

6. Conclusion

本報告根據作業說明實作並比較了 real-valued GA 與標準 PSO 在七個 benchmark functions 上的最佳化能力。

整體而言，PSO 在這組 benchmark 上表現顯著優於 GA。以 distance error（iter = 1000）衡量，PSO 在六個函數上勝出；Step 雖然 GA 的數值略低，但如 5.6 所述那個差距屬於平台效應造成的噪音，不代表真正的演算法優劣。在最困難的 Griewank 與 Kowalik 上，PSO 的 distance error 約為 GA 的四分之一。這個差距根本上來自兩個演算法截然不同的族群行為：

- PSO 透過 g_{best} 把整群粒子往好解拉，exploitation 能力強
- GA 每代靠 crossover 與 mutation 重新洗牌，族群多樣性高，但只能靠 elitism 帶著最佳解緩慢前進。

這種差異在 **Section 5.2** 的 sample run 圖與 **Section 5.4** 的 *Avg Mean Fitness* 欄位都清晰可見。

而困難函數也暴露了 GA 設計上的限制。固定的 BLX-α 搭配 Gaussian mutation 在 Rastrigin 等 2D 多峰函數上運作良好，但在 Griewank 密集的 local minima 與 Kowalik 的 4D 搜尋空間面前明顯吃力。另外，Griewank 的案例提醒我們 fitness 小不等於找到了真正的最佳解，評估演算法時單看 Table 4 的 best-so-far 容易低估問題難度，必須搭配 **Table 6** 的 distance error 一起判讀。

那部分指標的數值也受函數特性影響，所以不能直接解讀為演算法優劣。像是 Step 的 distance error 因平台效應天生偏高，Kowalik 的 *Avg Mean Fitness* 因分母爆炸而極不穩定，Penalized 的 GA *Avg Mean Fitness* 則被少數落到 $|x| > 5$ 高懲罰區域的個體拉爆。所以如果要讓我從這組實驗中選一個演算法，PSO 在大多數情況下是更穩健的選擇，不過 GA 的族群多樣性在需要廣域探索的問題上仍有其價值。